

Demo clasificador en “tiempo real”

Ejercicio: abducción-aducción de cadera.

Posición inicial: Maniquí acostado boca arriba, recto con pierna extendidas. Como se ve en la imagen artesanal de abajo.

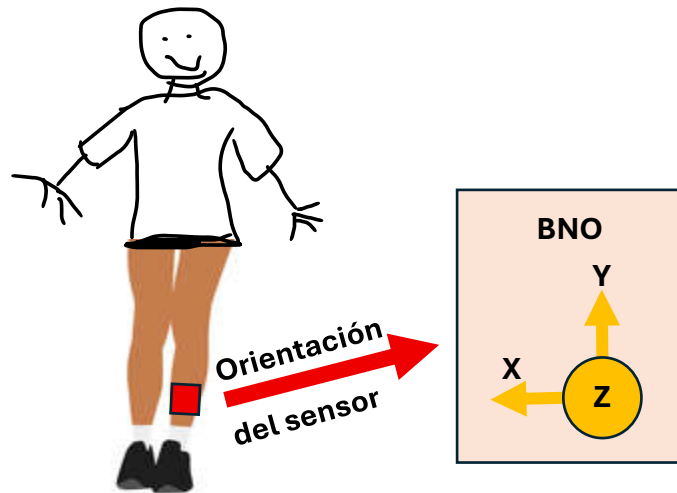


Figura 1. Posición inicial y orientación del nodo sensor.

Realización de ejercicio.

1. Partiendo de la posición inicial, se sujeta la pierna de maniquí (se puede sujetar del tobillo y del muslo) se levanta ligeramente la pierna para poder iniciar su desplazamiento hacia la derecha (movimiento de abducción) hasta llegar a aproximadamente 45°.
2. Regresar la pierna a la posición inicial.

Notas:

- Un ejercicio bien realizado se hace con movimientos suaves, continuos y estables.
- Las malas ejecuciones son ejercicios rápidos/bruscos o muy inestables.
- Parece que el clasificador se basa mas en el “tiempo de ejecución” que en el “ángulo” alcanzado. Sin embargo, abrir la pierna/cadera a ángulos mas grandes requiere de mas tiempo, y esta puede ser una razón por la cual el **segmentador rechace** esta ejecución. También llegar a ángulos grandes en poco tiempo hacen que la señal tenga mucha “área” y nuevamente sea rechazado por el **segmentador**.

CODIGO: imu_receiver_v2

Clasificador en: <https://github.com/MiG-Castro/codes> → c++ → imu_receiver_v2.exe

Para modificarlo usar `imu_receiver_v5_2.cpp`, abrir el CMD y compilarlo usando: `g++ imu_receiver_v5_2.cpp -o imu_receiver_v2.exe -lws2_32 -O2`

Nota: revisar que el nombre del .cpp coincida, lo subrayado en verde es el nombre que tendrá el ejecutable resultante.

El clasificador requiere:

- 1 nodo central (firmware **C10_ISO_n**)
- Al menos 1 nodo sensor con **BNO** (firmware **P10_ISO**).

Firmware de los nodos en: https://github.com/MiG-Castro/arts_mocap, `C/C10_ISO_n` y `P/P10_ISO`.

El firmware se compila usando la extensión NRF Connect de Visual Studio Code:

- SDK & Toolchain 3.0.1
- Board target: xiao_ble/nrf52840
- System build (sysbuild): No sysbuild

Al compilar se recomienda siempre usar la opción “pristine build”.

Preparación

- Ambos firmwares imprimen el address BLE del dispositivo al inicializar. El primer paso es obtener el address de los nodos involucrados. Para esto es necesario conectarlos mediante USB y ver los prints/logs en una terminal.
- En el firmware del nodo central añadir a la *white_list* el addr del nodo sensor, compilar y cargar al nodo central.

```
111 // Direcciones de TODOS Los dispositivos permitidos
112 // Definición de direcciones en Little Endian
113 // Original (BE) FA:53:60:43:10:2F -> LE=0x2F, 0x10, 0x43, 0x60, 0x53, 0xFA
114 static const bt_addr_le_t sensor_wl[] = {
115     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x2F, 0x10, 0x43, 0x60, 0x53, 0xFA}}},
116     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x86, 0xD4, 0x3C, 0x99, 0x28, 0xFA}}},
117     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x84, 0x20, 0x1F, 0x32, 0x67, 0xDC}}},
118     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x1B, 0xF8, 0xB5, 0x7C, 0xB0, 0xC0}}},
119     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x9B, 0xED, 0xDA, 0x2C, 0x1B, 0xEC}}},
120     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0xD0, 0x8D, 0x38, 0xBE, 0xC0, 0xCC}}},
121     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0xDA, 0x37, 0x2B, 0x40, 0x58, 0xD3}}},
122     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0x54, 0xB7, 0xDD, 0x0E, 0x29, 0xDD}}},
123     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0xA6, 0x35, 0xFD, 0x4E, 0xC2, 0xCE}}},
124     {.type = BT_ADDR_LE_RANDOM, .a = {.val = {0xFD, 0xC1, 0x75, 0x5F, 0x59, 0xEF}}},
125 };
```

- En el firmware del nodo sensor añadir a la *white_list* el addr del nodo central, compilar y cargar al nodo sensor.

```
51 // Dirección MAC del Broadcaster/Central Específico (Little Endian)
52 // FB:5C:2A:77:77:0A (Random) -> Pasar en lista en Big Endian
53 static const bt_addr_le_t target_addr = {
54     .type = BT_ADDR_LE_RANDOM, // 0-BT_ADDR_LE_PUBLIC si es pública
55     .a = {.val = {0xC3, 0xFB, 0xF6, 0xBA, 0xC4, 0xCD}}
56 };
57 // .a = {.val = {0x2C, 0xEB, 0x6B, 0x4F, 0x8A, 0xF3}}
58 // .a = {.val = {0xB3, 0x1C, 0x96, 0xB9, 0xCC, 0xE5}}
59 };
```

- Tras añadir las direcciones a ambos nodos, estos ahora serán capaces de sincronizarse y establecer links ACL automáticamente al encenderse.
- Conectar el nodo central a la computadora.
- Colocar el nodo sensor en el maniquí en su posición inicial y con la orientación como se muestra en la figura 1. Y prender el nodo sensor.
- Esperar unos 3-5 segundos para que se establezca la conexión. Si se quiere estar 100% seguro de que la conexión se estableció correctamente se puede revisar los logs del nodo central en una terminal. Al establecerse la conexión el nodo central debe de imprimir los siguientes mensajes. Al terminar hay que desconectar/liberar el puerto serial.

```
Serial Terminal v1.6.1
USB-DEV 080E98D891D7D2F0
TERMINAL SETTINGS ABOUT

SERIAL PORT
COM11
Disconnect from port

SERIAL SETTINGS
TERMINAL MODE
WRITE TO FILE

> Type and press Enter to send

*** Booting nRF Connect SDK v3.0.1-9eb5615da66b ***
*** Using Zephyr OS v4.0.9-7f865b8f8d0 ***
Central_N C6:9B:47:DE:A4:AF (random). CEL=2000us, ISO_Interval=200ms
ISO C[0x200006d0] connected
[C] Connected to: DC:67:32:1F:20:84 (random).
- PHY set to 1M
- Connection parameters set to 10.00 ms, 3, 1000 ms.
- Data Length (Tx/Rx): 70/70 bytes, 666/666 us.
- MTU set to 73 bytes.
[P0] subscription completed.
```

- Ejecutar el **imu_receiver_v2** desde el CMD. El código espera recibir de argumento el COM del nodo centra. Si no se añade el argumento, el código intentara conectarse al COM7. La siguiente figura muestra como correr el ejecutable indicando el puerto. REVISAR QUE EL NOMBRE DEL EJECUTABLE Y EL COM SELECCIONADO SEA EL CORRECTO.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\MACP_\Documents\Codigos\c++> .\imu_receiver_v2.exe COM11
```

- Dejar quieto el nodo sensor hasta recibir el mensaje de “Rotation Matrix Ready”

```
=== IMU Receiver System (Windows/MinGW) ===
Puerto: COM11
Baudrate: 230400
Perifericos: 4
Buffer: 768 muestras/periferico
Segmentador: IAAC en Nodo 0
=====
```

```
Puerto serial COM11 abierto a 230400 baud
UDP Sender configurado para 127.0.0.1:4000
Min PKT len: 67
```

```
=== TX START CMD's ===
TX-CMD: 7E 00 18 7F
TX-CMD: 7E 00 17 7F
TX-CMD: 7E 03 00 FF 03 01 7F
Threads iniciados (RX + Process)
```

```
=== Sistema en ejecucion ===
Presiona ENTER para detener y cerrar archivo...
[Process Thread] Iniciado - Segmentador IAAC Periferico 0
[RX Thread] Iniciado
[RX] Archivo abierto: 2026-07-16_16-03-55.bin
[ED] P0 Rot-Matrix Ready
```

- El sistema espera recibir 60 muestras del nodo sensor para generar la matriz de rotación. Una vez se genere la matriz de rotación el sistema estará listo para usarse.
- Al finalizar presionar la tecla enter para enviar los comandos de “stop” al nodo sensor y terminar el programa.

```
TX STOPS CMD's
TX-CMD: 7E 03 00 FF 03 00 7F
TX-CMD: 7E 00 16 7F

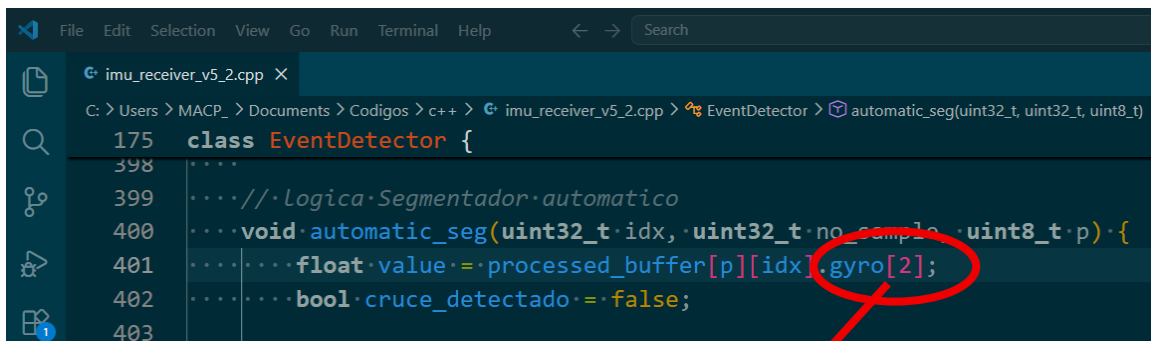
Stop threads...
[RX] Cerrando archivo...
[RX] Archivo cerrado: 607 paquetes escritos
[RX Thread] Terminado
[Process Thread] Terminado

=== ESTADISTICAS ===
Archivo: 2026-07-16_17-09-13.bin
Paquetes raw escritos: 607
  P0: 607 paquetes recibidos, 0 perdidos (0.00%)
  P1: 0 paquetes recibidos, 0 perdidos
  P2: 0 paquetes recibidos, 0 perdidos
  P3: 0 paquetes recibidos, 0 perdidos

Sistema detenido. Archivo cerrado.
Puerto serial cerrado
```

Notas:

- **Para la segmentación y clasificación solo se usan los datos del primer nodo sensor conectado.** Por lo que, aunque se pueden conectar mas nodos sensores, son irrelevantes para esta versión del código.
- El segmentador puede rechazar una ejecución si se realiza en muy poco tiempo o mucho tiempo.
- **Por alguna razón la matriz de rotación no esta funcionando correctamente, así que es NECESARIO RESPETAR LA ORIENTACION DE LA FIGURA 1.**
- **Si el código empieza a fallar reiniciar (incluyendo el nodo sensor). Si ni con eso funciona y en general parece que se niega a detectar los movimientos, el siguiente paso es cambiar la señal de referencia del segmentador.** En el repositorio hay 2 ejecutables el "..._gy.exe" y el "..._gz.exe". Estos son las 2 señales que me han funcionado. Si no funciona uno intentar con el otro y sino moverle al código.



```
175 class EventDetector {
398     ...
399     ...//logica Segmentador automatico
400     ...void automatic_seg(uint32_t idx, uint32_t no_sample, uint8_t p){
401         ...float value = processed_buffer[p][idx].gyro[2];
402         ...bool cruce_detectado = false;
403     }
```

gyro[0] = giroscopio eje x
gyro[1] = giroscopio eje y
gyro[2] = giroscopio eje z

XX

En nuevas versiones "rt_classf_XXXX", muestran la validación de los segmentos del ejercicio: **X**, invalido; **[+]**, segmento "positivo" valido y; **[-]**, segmento negativo valido. La detección de un posible ejercicio consiste en encontrar la secuencia **[+]** y **[-]** en una determinada ventana de tiempo, si no se cumple esa ventana, aunque se encuentre la secuencia estará "fuera de rango" y será invalida.

```
Windows PowerShell

=== TX START CMD's ===
TX-CMD: 7E 00 18 7F
TX-CMD: 7E 00 17 7F
TX-CMD: 7E 03 00 FF 03 01 7F
Threads iniciados (RX + Process)

=== Sistema en ejecucion ===
Presiona ENTER para detener y cerrar archivo...
[Process Thread] Iniciado - Segmentador IAAC Periferico 0
[RX Thread] Iniciado
[RX] Archivo abierto: 2026-07-22_00-24-30.bin
[ED] P0 Rot-Matrix Ready
XXXXXXXXXX[+]X[-]
Possible exercise: Well-performed
X[+][-]
Possible exercise: Poorly performed
X[+][-]
Possible exercise: Poorly performed
X[+]XXX[+]X[-]
Possible exercise: Well-performed
X[+]X[-]
Possible exercise: Poorly performed
[-][+][-]
Possible exercise: Poorly performed
X[+]X[-]
Possible exercise: Poorly performed
[-][-]XXX[+]X[-]
Possible exercise: Poorly performed
XXXXXXXXXX
```

Para la versión “rt_classf_PS” (PS=PlaySound), esta reproduce sonido en la ejecución correcta y otro en la “poorly”.

La versión “rt_classf” se basa en el código v5_3 y se compila con el comando mostrado anteriormente.

La versión “rt_classf_PS” se basa en el código v5_4 y se compila con este comando:
g++ imu_receiver_v5_4.cpp -o rt_classf_PS_gy.exe -lws2_32 -lwinmm -O2

En caso de que se quiera eliminar la visualización de la validación de las secciones del ejercicio, se puede comentar la siguiente parte del código y compilar:

```
189 class EventDetector {
414     void automatic_seg(uint32_t idx, uint32_t no_sample, uint8_t p) {
440         else {
446             } else {
463             if(!v) std::cout<<RED<<"X"<<RST<<std::flush;
464             else{
465                 if(seg_actual.sign==1) std::cout<<GRN<<"["<<RST<<std::flush;
466                 else std::cout<<GRN<<"["<<RST<<std::flush;
467             }
468 }
```